

# TP4 – Scraping web : BeautifulSoup, Scrapy et Playwright

M1 Data & IA – Université Catholique de Lille

## Rendu attendu

- Notebook Jupyter `.ipynb` exécutable de bout en bout.
- Fichiers de sortie : `communes_wiki.parquet`, `communes_scrapy.json`, `communes_scrapy.parquet`, `dvf_enrichi_scraping.parquet`, `dvf_carte_finale.html`.
- Fichier `geocache.json` du TP3 (si présent).
- Travail en binôme conseillé.
- Nom du fichier : `TP4_Scraping_NOM_Prenom.ipynb`

## Contexte

Votre responsable à la foncière lilloise veut aller plus loin. Les données DVF et l'API INSEE intégrées en TP3 donnent une vision macroéconomique solide, mais il manque des informations qualitatives sur chaque commune : canton, arrondissement, intercommunalité, superficie, densité – toutes disponibles dans des tableaux HTML sur Wikipedia.

Ce TP construit une troisième couche d'enrichissement par scraping. Trois outils successifs : **BeautifulSoup** pour les pages statiques, **Scrapy** pour une architecture Spider, **Playwright** pour le contenu dynamique.

Le dataset de référence est `dvf_enrichi_final.parquet` issu du TP3.

## Sources de données et outils

- Dataset fil rouge : `dvf_enrichi_final.parquet` (TP3).
- Source 1 & 2 (BS4 + Scrapy) : [https://fr.wikipedia.org/wiki/Liste\\_des\\_communes\\_du\\_Nord](https://fr.wikipedia.org/wiki/Liste_des_communes_du_Nord)
- Source 3 (Playwright) : même page Wikipedia, pour comparer requests vs Playwright.
- Outils : `requests`, `beautifulsoup4`, `lxml`, `scrapy`, `playwright`, `pydantic`, `pandas`, `folium`, `unidecode`.

## Objectifs pédagogiques

À l'issue du TP, vous devez être capables de :

- lire un `robots.txt` et décider légalement si une source est scrapable ;
- extraire un tableau HTML avec BeautifulSoup et le transformer en DataFrame ;
- écrire un Spider Scrapy minimal avec Item Pipeline Pydantic ;
- utiliser Playwright pour extraire du contenu rendu par JavaScript ;
- intégrer le scraping dans le pipeline DVF via une jointure normalisée ;
- comparer les trois approches en simplicité, robustesse et performance.

## 1. Mise en place

### 1.1. Installations

```
pip install requests beautifulsoup4 lxml scrapy playwright pydantic pandas folium tqdm
unidecode
playwright install chromium
```

### 1.2. Imports et configuration

```
import requests
import time, random, json, os, warnings, re
import pandas as pd
import numpy as np
from bs4 import BeautifulSoup
from pydantic import BaseModel, Field, field_validator, ValidationError
from typing import Optional
from pathlib import Path
from tqdm import tqdm
import folium
import unidecode

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 50)

SESSION = requests.Session()
SESSION.headers.update({
    "User-Agent": "UCLille-TP4/1.0 (votre@email.ucl.fr)",
    "Accept-Language": "fr-FR,fr;q=0.9",
    "Accept": "text/html,application/xhtml+xml",
})
```

### 1.3. Chargement du dataset

#### Dataset requis

Ce TP utilise uniquement `dvf_enrichi_final.parquet` produit en TP3. Si vous ne l'avez pas, relancez la question 20 du TP3 avant de continuer.

```
if not Path("dvf_enrichi_final.parquet").exists():
    raise FileNotFoundError(
        "dvf_enrichi_final.parquet introuvable. "
        "Relancer le pipeline TP3 (question 20) avant de continuer."
    )

df = pd.read_parquet("dvf_enrichi_final.parquet")
apparts = df[df["type_local"] == "Appartement"].copy()
```

#### Question 1. Vérification légale.

```
def lire_robots(base_url):
    r = SESSION.get(f"{base_url}/robots.txt", timeout=10)
    print(f"=== robots.txt de {base_url} ===")
    print(r.text[:1500])
```

```
return r.text

robots_wiki = lire_robots("https://fr.wikipedia.org")
```

1. Wikipedia définit-il un `Crawl-delay`? Quelle valeur de délai adopter?
2. Le scraping des pages `/wiki/` est-il autorisé? Quelles zones sont interdites?
3. Rédigez une décision documentée pour cette source.

## 2. BeautifulSoup – Pages statiques

### 2.1. Récupération de la page

```
URL_WIKI = "https://fr.wikipedia.org/wiki/Liste_des_communes_du_Nord"

r = SESSION.get(URL_WIKI, timeout=15)
r.raise_for_status()
r.encoding = "utf-8"

soup = BeautifulSoup(r.text, "lxml")
```

**Question 2.** Inspectez la structure.

```
tables = soup.find_all("table", class_="wikitable")

for i, t in enumerate(tables[:3]):
    headers = [th.get_text(strip=True) for th in t.find_all("th")[:10]]
    n_rows = len(t.find_all("tr"))
    print(f"Table {i}: {n_rows} lignes -- enttes : {headers}")

table_principale = tables[0]
rowspan_count = len(table_principale.find_all(attrs={"rowspan": True}))
colspan_count = len(table_principale.find_all(attrs={"colspan": True}))
print(f"Cellules rowspan : {rowspan_count}, colspan : {colspan_count}")
```

1. Combien de tableaux `wikitable`? Lequel est le principal?
2. Listez les colonnes du tableau principal et indiquez le type attendu de chacune.
3. La colonne **Population** contient une année entre parenthèses (ex : « 238 695 (2022) »). Comment l'extraire en nombre?

### 2.2. Extraction du tableau

```
def extraire_table_wikitable(soup, index=0):
    """Extrait le index-ime tableau wikitable en DataFrame."""
    tables = soup.find_all("table", class_="wikitable")
    if not tables:
        raise ValueError("Aucun tableau wikitable trouv")
    table = tables[index]

    headers = [th.get_text(strip=True) for th in table.find_all("th")]
    seen, unique = {}, []
    for h in headers:
        seen[h] = seen.get(h, 0) + 1
```

```

        unique.append(f"{h}_{seen[h]}" if seen[h] > 1 else h)

rows = []
tbody = table.find("tbody") or table
for tr in tbody.find_all("tr"):
    cells = tr.find_all("td")
    if not cells:
        continue
    row = [c.get_text(separator=" ", strip=True) for c in cells]
    if row and any(row):
        rows.append(row)

n_cols = len(unique)
rows = [r[:n_cols] + [""] * max(0, n_cols - len(r)) for r in rows]
return pd.DataFrame(rows, columns=unique[:n_cols])

df_wiki_raw = extraire_table_wikitable(soup, index=0)
df_wiki_raw.head()

```

**Question 3.** Nettoyez et typez le DataFrame.

```

def nettoyer_texte_wiki(s):
    """Retire les rfrences [1], les espaces multiples et '(prfecture)'."""
    s = re.sub(r"\[d+\]", "", str(s))
    s = re.sub(r"\s+", " ", s).strip()
    s = re.sub(r"\s*\([~]*\)s*$", "", s)
    return s

def nettoyer_nombre(s):
    """Convertit '238 695 (2022)' ou '34,83' en float."""
    if not s or str(s) in ["-", "N/A", "", ""]:
        return None
    s = re.sub(r"\([~]*\)", "", str(s))
    s_clean = re.sub(r"^\d,.", "", s).replace(",", ".")
    try:
        return float(s_clean)
    except ValueError:
        return None

df_wiki = df_wiki_raw.copy()
df_wiki["Nom"] = df_wiki["Nom"].apply(nettoyer_texte_wiki)

col_pop = [c for c in df_wiki.columns if "Population" in c][0]
df_wiki["population_num"] = df_wiki[col_pop].apply(nettoyer_nombre)

if "Superficie (km2)" in df_wiki.columns:
    df_wiki["superficie_num"] = df_wiki["Superficie (km2)"].apply(nettoyer_nombre)

```

1. Combien de communes contient le DataFrame ? Est-ce cohérent avec le nombre attendu ?
2. Quel est le taux de conversion de la population ? Si non parfait, quelles lignes posent problème ?
3. Identifiez les 5 communes les plus peuplées. Correspondent-elles à ce que vous attendiez ?

## 2.3. Jointure avec DVF

```
def normalize(s):
    if pd.isna(s) or not s:
        return ""
    return unicode.unidecode(str(s)).upper().strip().replace("-", " ")

df_wiki["cle"] = df_wiki["Nom"].apply(normalize)
apparts["cle_wiki"] = apparts["nom_commune"].apply(normalize)

df_wiki_dedup = df_wiki.drop_duplicates(subset="cle")

cols_utiles = ["cle", "Nom", "population_num"]
for col_opt in ["Arrondissement", "Intercommunalit"]:
    if col_opt in df_wiki.columns:
        cols_utiles.append(col_opt)

apparts = pd.merge(
    apparts, df_wiki_dedup[cols_utiles],
    left_on="cle_wiki", right_on="cle", how="left",
    suffixes=("", "_wiki"),
)
```

#### Question 4.

1. Quel est le taux de correspondance ? Comparez-le à celui obtenu via l'API INSEE en TP3.
2. Listez les 5 communes DVF non appariées et proposez une cause pour chacune.
3. Pourquoi utilise-t-on `.replace("-", " ")` dans la normalisation ? Donnez un exemple où cela aide la jointure et un cas où cela pourrait créer un faux positif.

#### 2.4. Validation Pydantic

```
class CommuneWiki(BaseModel):
    cle: str
    nom: str
    population: Optional[float] = None

    @field_validator("cle")
    @classmethod
    def cle_non_vide(cls, v):
        if not v.strip():
            raise ValueError("Cl vide")
        return v

valides, rejets = [], []
for _, row in df_wiki_dedup.iterrows():
    try:
        c = CommuneWiki(
            cle=str(row["cle"]),
            nom=str(row["Nom"]),
            population=row["population_num"],
        )
        valides.append(c.model_dump())
    except ValidationError as e:
        rejets.append({"nom": row.get("Nom"), "erreur": str(e)})
```

```
df_communes_wiki = pd.DataFrame(valides)
df_communes_wiki.to_parquet("communes_wiki.parquet", index=False)
```

**Question 5.** À quoi sert de passer par Pydantic pour valider des données Wikipedia ? Citez deux types d'erreurs que Pydantic détecterait sur ce dataset.

### 3. Scrapy – Scraping avec Spider

#### Note Scrapy en notebook

Scrapy n'est pas conçu pour Jupyter (conflit asyncio). On écrit le spider dans un .py et on le lance depuis une cellule via `subprocess`.

#### 3.1. Analyse avec Scrapy Shell

**Question 6.** Dans un terminal :

```
scrapy shell "https://fr.wikipedia.org/wiki/Liste_des_communes_du_Nord"

>>> response.css("table.wikitable")
>>> response.css("table.wikitable tbody tr").getall()[:2]
>>> response.css("table.wikitable tbody tr td::text").getall()[:10]
```

1. Combien de `<tr>` contient le tableau ?
2. Combien de `<td>` par ligne ?
3. Quel sélecteur CSS extrairait uniquement le nom de la commune (premier `<td>` de chaque ligne) ?

#### 3.2. Spider complet

**Question 7.** Écrire le spider.

```
%%writefile communes_spider.py
import scrapy

class CommuneItem(scrapy.Item):
    nom = scrapy.Field()
    code_insee = scrapy.Field()
    arrondissement = scrapy.Field()
    intercommunalite = scrapy.Field()
    superficie = scrapy.Field()
    population = scrapy.Field()
    source_url = scrapy.Field()

class CommunesSpider(scrapy.Spider):
    name = "communes_nord"
    start_urls = ["https://fr.wikipedia.org/wiki/Liste_des_communes_du_Nord"]

    custom_settings = {
        "DOWNLOAD_DELAY": 2,
        "RANDOMIZE_DOWNLOAD_DELAY": True,
        "ROBOTSTXT_OBEY": True,
```

```

"AUTOTHROTTLE_ENABLED": True,
"LOG_LEVEL": "INFO",
"USER_AGENT": "UCLille-TP4/1.0 (etudiant@ucl.fr)",
"FEEDS": {
    "communes_scrapy.json": {
        "format": "json", "encoding": "utf8", "overwrite": True
    }
},
}

def parse(self, response):
    # Structure : Nom / Code Insee / Code postal / Arrondissement / Canton
    # / Intercommunalit / Superficie / Population / Densit / Modifier
    for tr in response.css("table.wikitable tbody tr"):
        cells = tr.css("td")
        if len(cells) < 8:
            continue

        nom = cells[0].css("a::text").get(default="") or cells[0].css("::text").get(
            default="")

        yield CommuneItem(
            nom=nom.strip(),
            code_insee=cells[1].css("::text").get(default="").strip(),
            arrondissement=cells[3].css("a::text, ::text").get(default="").strip(),
            intercommunalite=cells[5].css("a::text, ::text").get(default="").strip(),
            superficie=cells[6].css("::text").get(default="").strip(),
            population=cells[7].css("::text").get(default="").strip(),
            source_url=response.url,
        )

```

### 3.3. Lancement du spider

```

import subprocess

if Path("communes_scrapy.json").exists():
    Path("communes_scrapy.json").unlink()

result = subprocess.run(
    ["scrapy", "runspider", "communes_spider.py",
     "-o", "communes_scrapy.json"],
    capture_output=True, text=True, timeout=120
)
print("Code retour :", result.returncode)
if result.stderr:
    for ligne in result.stderr.strip().split("\n")[-15:]:
        print(ligne)

```

### 3.4. Post-traitement

```

with open("communes_scrapy.json", encoding="utf-8") as f:
    data_scrapy = json.load(f)

df_scrapy = pd.DataFrame(data_scrapy)

```

```
# Important : reutiliser nettoyer_nombre() de la partie 1 pour gerer
# le format Wikipedia "238 695 (2022)"
df_scrapy["population_num"] = df_scrapy["population"].apply(nettoyer_nombre)
df_scrapy["superficie_num"] = df_scrapy["superficie"].apply(nettoyer_nombre)
```

### Question 8.

1. Combien d'items le spider a-t-il collectés ?
2. Dans `result.stderr`, repérez `item_scraped_count` et `downloader/request_count`. Combien de requêtes ont été envoyées ?
3. Comparez les colonnes Scrapy avec celles du DataFrame BS4 (partie 1). Laquelle des deux méthodes est la plus simple à mettre en œuvre pour cette tâche ?

## 3.5. Item Pipeline avec Pydantic

**Question 9.** Ajouter un pipeline de validation au spider.

```
%%writefile communes_spider_v2.py
import scrapy
from pydantic import BaseModel, field_validator
from typing import Optional
import re
import pandas as pd

def nettoyer_nombre(s):
    if not s or str(s) in ["-", "N/A", "", ""]:
        return None
    s = re.sub(r"\([^)]*\)", "", str(s))
    s_clean = re.sub(r"[^\\d,\\.]", "", s).replace(",", ".")
    try:
        return float(s_clean)
    except ValueError:
        return None

class CommuneValidee(BaseModel):
    nom: str
    code_insee: str
    population: Optional[float] = None
    source_url: str

    @field_validator("nom")
    @classmethod
    def nom_non_vide(cls, v):
        if not v.strip():
            raise ValueError("Nom vide")
        return v.strip()

    @field_validator("code_insee")
    @classmethod
    def code_dept_59(cls, v):
        if not v.startswith("59"):
            raise ValueError(f"Code INSEE invalide : {v}")
```



```

        return v

class ValidationPipeline:
    def __init__(self):
        self.items = []
        self.rejets = []

    def process_item(self, item, spider):
        try:
            pop_num = nettoyer_nombre(item.get("population"))
            c = CommuneValidee(
                nom=str(item.get("nom", "")),
                code_insee=str(item.get("code_insee", "")),
                population=pop_num,
                source_url=str(item.get("source_url", "")),
            )
            self.items.append(c.model_dump())
        except Exception as e:
            self.rejets.append({"item": dict(item), "erreur": str(e)})
        return item

    def close_spider(self, spider):
        df = pd.DataFrame(self.items)
        df.to_parquet("communes_scrapy.parquet", index=False)
        spider.logger.info(
            f"Pipeline : {len(self.items)} valides, {len(self.rejets)} rejetes"
        )

class CommuneItem(scrapy.Item):
    nom = scrapy.Field()
    code_insee = scrapy.Field()
    population = scrapy.Field()
    source_url = scrapy.Field()

class CommunesSpiderV2(scrapy.Spider):
    name = "communes_v2"
    start_urls = ["https://fr.wikipedia.org/wiki/Liste_des_communes_du_Nord"]
    custom_settings = {
        "DOWNLOAD_DELAY": 2,
        "ROBOTSTXT_OBEY": True,
        "AUTOTHROTTLE_ENABLED": True,
        "LOG_LEVEL": "INFO",
        "USER_AGENT": "UCLille-TP4/1.0 (etudiant@ucl.fr)",
        "ITEM_PIPELINES": {"communes_spider_v2.ValidationPipeline": 100},
    }

    def parse(self, response):
        for tr in response.css("table.wikitable tbody tr"):
            cells = tr.css("td")
            if len(cells) < 8:
                continue
            nom = cells[0].css("a::text").get(default="") or cells[0].css("::text").get(

```

```

        default="")
    yield CommuneItem(
        nom=nom.strip(),
        code_insee=cells[1].css("::text").get(default="").strip(),
        population=cells[7].css("::text").get(default="").strip(),
        source_url=response.url,
    )

```

```

result = subprocess.run(
    ["scrapy", "runspider", "communes_spider_v2.py"],
    capture_output=True, text=True, timeout=120
)

df_scrapy_propre = pd.read_parquet("communes_scrapy.parquet")

```

Combien d'items ont été validés ? Combien rejetés ? Quelle règle a entraîné les rejets ?

## 4. Playwright – Contenu dynamique

### 4.1. Diagnostic : la page est-elle dynamique ?

#### Choix pédagogique de l'URL

Pour cette section, on **réutilise** l'URL de la partie 1. Comme c'est une page statique, **requests** et Playwright donnent le même résultat. C'est précisément l'objectif : démontrer que Playwright n'est pas toujours nécessaire et a un coût en performance.

```

URL_DYN = URL_WIKI

r_test = SESSION.get(URL_DYN, timeout=15)
soup_test = BeautifulSoup(r_test.text, "lxml")
tables_req = soup_test.find_all("table", class_="wikitable")
print(f"Tables trouvées avec requests : {len(tables_req)}")

```

#### Question 10.

1. Combien de tables **requests** a-t-il trouvées ? La page est-elle statique ou dynamique ?
2. Citez deux types de pages où **requests** échoue totalement (HTML source vide ou squelette).
3. Donnez un exemple concret de site français qui nécessite Playwright.

### 4.2. Scraping avec Playwright

#### Pièges Playwright à éviter

- `wait_for_selector("table.wikitable")` timeout si la page n'a pas de wikitable. Toujours wrapper dans un `try/except`.
- `wait_for_load_state("networkidle")` peut hanger sur des pages avec websockets ou analytics. Préférer `wait_until="domcontentloaded"` dans `goto()`.
- Dans le `except`, `page.screenshot()` échoue parfois. Toujours wrapper dans un second `try/except`.

```

from playwright.sync_api import sync_playwright

def scraper_page_dynamique(url, selecteur_attente=None, timeout_ms=20_000):
    """
    Charge une page avec Playwright et retourne le HTML.

    - selecteur_attente : optionnel, attend qu'un element CSS apparaisse
    - Si le selecteur n'apparait pas dans les 10s, on continue quand meme
    - Pas de wait_for_load_state(networkidle) qui hange sur Wikipedia
    """
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        context = browser.new_context(
            user_agent="UCLille-TP4/1.0 (etudiant@ucl.fr)",
            locale="fr-FR",
        )
        page = context.new_page()
        try:
            page.goto(url, timeout=timeout_ms, wait_until="domcontentloaded")

            if selecteur_attente:
                try:
                    page.wait_for_selector(selecteur_attente, timeout=10_000)
                except Exception:
                    print(f"Selecteur '{selecteur_attente}' non trouve, on continue")

            page.wait_for_timeout(1_000)
            return page.content()

        except Exception as e:
            print(f"Erreur Playwright : {type(e).__name__}: {str(e)[:200]}")
            try:
                page.screenshot(path="debug_playwright.png")
            except Exception:
                pass
            return ""
        finally:
            browser.close()

t0 = time.perf_counter()
html_dynamique = scraper_page_dynamique(URL_DYN, "table.wikitable")
t_playwright = time.perf_counter() - t0
print(f"Temps Playwright : {t_playwright:.1f}s")

```

**Question 11.** Extraire et comparer.

```

if html_dynamique:
    soup_dyn = BeautifulSoup(html_dynamique, "lxml")
    tables_dyn = soup_dyn.find_all("table", class_="wikitable")

    if tables_dyn:
        df_playwright = extraire_table_wikitable(soup_dyn, index=0)

```

1. Les deux DataFrames (BS4 vs Playwright) ont-ils la même taille? Pourquoi?

2. Mesurez le ratio `t_playwright` / `t_bs4`. Combien de fois Playwright est-il plus lent ?
3. Pour quelle raison pourrait-on quand même utiliser Playwright sur une page statique ?

**Question 12.** Pour chacun des trois scénarios, indiquez l'outil approprié (**requests+BS4**, **Playwright+BS4** ou **Playwright+interception réseau**) et justifiez :

1. Site institutionnel avec un tableau HTML statique mis à jour mensuellement.
2. Tableau de bord React qui charge les données via `fetch('/api/indicateurs')`.
3. Page qui affiche les données après un clic sur « Afficher les résultats ».

## 5. Pipeline enrichi et carte finale

### 5.1. Pipeline

```
def pipeline_enrichi_scraping(df_source):
    df = df_source[df_source["type_local"] == "Appartement"].copy()
    df["prix_m2"] = df["valeur_fonciere"] / df["surface_reelle_bati"]
    df = df[df["prix_m2"].between(500, 15_000)].copy()

    df["cle_commune"] = df["nom_commune"].apply(normalize)

    if Path("communes_wiki.parquet").exists():
        df_w = pd.read_parquet("communes_wiki.parquet")
        df_w["cle"] = df_w["nom"].apply(normalize)
        df_w_dedup = df_w.drop_duplicates("cle")
        df = pd.merge(
            df,
            df_w_dedup[["cle", "population"]].rename(columns={"population": "pop_wiki"}),
            left_on="cle_commune", right_on="cle", how="left"
        )

    if Path("communes_scrapy.parquet").exists():
        df_s = pd.read_parquet("communes_scrapy.parquet")
        df_s["cle_s"] = df_s["nom"].apply(normalize)
        df_s_dedup = df_s.drop_duplicates("cle_s")
        df = pd.merge(
            df,
            df_s_dedup[["cle_s", "population"]].rename(columns={"population": "pop_scrapy"}),
            left_on="cle_commune", right_on="cle_s", how="left"
        )

    cache_geo = {}
    if Path("geocache.json").exists():
        cache_geo = json.loads(Path("geocache.json").read_text(encoding="utf-8"))

    if "adresse_complete" in df.columns:
        df["lat"] = df["adresse_complete"].map(
            lambda a: cache_geo.get(a, {}).get("lat") if isinstance(cache_geo.get(a), dict)
            else None
        )
        df["lon"] = df["adresse_complete"].map(
            lambda a: cache_geo.get(a, {}).get("lon") if isinstance(cache_geo.get(a), dict)
            else None
        )
```

```

    )

    df.to_parquet("dvf_enrichi_scraping.parquet", index=False)
    return df

df_final = pipeline_enrichi_scraping(df)

```

**Question 13.** Complétez le tableau de couverture avec vos chiffres réels.

| Source                     | Méthode       | Couverture (%) | Remarque |
|----------------------------|---------------|----------------|----------|
| Population Wikipedia (BS4) | BeautifulSoup | ???            |          |
| Population Scrapy          | Scrapy        | ???            |          |
| Coordonnées GPS            | Cache TP3     | ???            |          |
| <b>Les trois à la fois</b> | Combined      | ???            |          |

Quelle source offre la meilleure couverture? Les sources BS4 et Scrapy donnent-elles des chiffres cohérents pour la population?

## 5.2. Carte interactive avec couche agrégée

**Question 14.**

```

df_carte = df_final.dropna(subset=["lat", "lon", "prix_m2"]).copy()

if len(df_carte) > 0:
    centre = [df_carte["lat"].median(), df_carte["lon"].median()]
    carte = folium.Map(location=centre, zoom_start=11, tiles="CartoDB positron")

    q1 = df_carte["prix_m2"].quantile(0.25)
    med = df_carte["prix_m2"].quantile(0.50)
    q3 = df_carte["prix_m2"].quantile(0.75)

    def couleur(p):
        if p < q1: return "#15803D"
        if p < med: return "#1E8FA0"
        if p < q3: return "#D97706"
        return "#B91C1C"

    agg = (df_carte.groupby("nom_commune")
           .agg(n=("prix_m2", "count"),
                p_med=("prix_m2", "median"),
                lat_med=("lat", "median"),
                lon_med=("lon", "median"),
                pop=("pop_wiki", "first"))
           .query("n >= 10").reset_index())

    if len(agg) > 1:
        r_min, r_max = 5, 20
        agg["rayon"] = r_min + (agg["n"] - agg["n"].min()) / \
            max(agg["n"].max() - agg["n"].min(), 1) * (r_max - r_min)
    else:
        agg["rayon"] = 10

    for _, row in agg.iterrows():
        pop = f"{row['pop']:,0f} hab." if pd.notna(row["pop"]) else "N/A"

```

```

folium.CircleMarker(
    location=[row["lat_med"], row["lon_med"]],
    radius=row["rayon"],
    color=couleur(row["p_med"]),
    fill=True, fill_opacity=0.65, weight=1.5,
    popup=folium.Popup(
        f"<b>{row['nom_commune']}

```

Quelles communes apparaissent en cercles rouges (prix les plus élevés) ? Quelles communes en vert ?  
Le pattern géographique correspond-il à votre connaissance de la métropole lilloise ?

## 6. Analyse des biais et qualité

**Question 15.** Cohérence Wikipedia vs INSEE.

```

if "pop_commune" in df_final.columns and "pop_wiki" in df_final.columns:
    comp = (df_final.dropna(subset=["pop_commune", "pop_wiki"])
            .groupby("nom_commune")
            .agg(pop_insee=("pop_commune", "first"),
                 pop_wiki=("pop_wiki", "first"))
            .reset_index())
    comp["ecart_pct"] = ((comp["pop_wiki"] - comp["pop_insee"])
                        / comp["pop_insee"].replace(0, np.nan) * 100).round(1)
    print(f"Communes comparees : {len(comp)}")
    print(f"Ecart median : {comp['ecart_pct'].median():.1f} %")
    print(comp.nlargest(5, "ecart_pct")[["nom_commune", "pop_insee", "pop_wiki", "ecart_pct"]])

```

1. L'écart médian est-il faible ( $< 5\%$ ) ? À quoi est-il dû ?
2. Les communes avec le plus grand écart sont-elles des cas particuliers ?
3. Pour la production, quelle source privilégier et pourquoi ?

**Question 16.** Analyse des biais en trois points :

1. **Biais de couverture** : quelles communes sont sous-représentées dans Wikipedia ?
2. **Biais de fraîcheur** : les données peuvent-elles être périmées ? Comment vérifier ?
3. **Biais de sélection** : enrichir uniquement les communes présentes dans DVF et Wikipedia introduit-il un biais ?

**Question 17.** Benchmark.

```

t0 = time.perf_counter()
r = SESSION.get(URL_WIKI, timeout=15)
soup_b = BeautifulSoup(r.text, "lxml")
_ = extraire_table_wikitable(soup_b, 0)
t_bs4 = time.perf_counter() - t0

```

```
print(f"requests + BS4 : {t_bs4:.2f}s")
print(f"Playwright + BS4 : {t_playwright:.2f}s")
if t_bs4 > 0:
    print(f"Ratio : x{t_playwright/t_bs4:.1f}")
```

Le surcoût de Playwright est-il justifié pour une page statique ? Quel est le critère décisif pour choisir entre les deux ?

## 7. Rapport technique (optionnel)

**Question 18.** Rédigez un rapport à votre responsable, structuré ainsi :

1. **Apport du scraping** : quelles données nouvelles le pipeline intègre-t-il par rapport au TP3 ?
2. **Fiabilité comparée** : comment les données scrapées se comparent-elles à celles de l'API INSEE ?
3. **Coût de maintenance** : quel effort si Wikipedia change la structure de la page ?

Le rapport doit être compréhensible par une personne qui ne code pas.

### Conseils de présentation

- Structurez le notebook avec les titres des sections.
- Pour chaque question, rédigez une cellule Markdown de réponse sous le code.
- Incluez les fichiers de sortie dans le rendu.
- Vérifiez que le notebook s'exécute de bout en bout sans erreur.